
Neural Mesh Architecture

Squirrel OS & JasperOS

Technical Reference | Public Release Edition

Document Version:	v1.1 — Public Release
Classification:	Open — Ecosystem Reference
Issued:	25 July 2026
Patent References:	U.S. Provisional 64/114,746 · U.S. Patent Application 19/693,343
Inventor:	Leon Calvin Long II
Companion Document:	JasperOS HOWTO Manual

1. Abstract

The Neural Mesh Architecture is the adaptive intelligence component of Squirrel OS, engineered to deliver self-healing capability across a fleet of 150 or more fintech applications deployed on the Base44 platform. Rather than employing conventional neural network constructs — matrix multiplication, gradient descent, or backpropagation through calculus — this architecture instantiates network topology as persistent database entity records (NeuralNode) and delegates all forward propagation and weight adjustment to a Large Language Model (LLM) acting as the compute engine. The result is a network that survives process restarts, remains fully queryable, replicates deterministically across application instances, and reasons adaptively over novel anomaly types without retraining.

The architecture operates beneath JasperOS, a deterministic runtime governance layer that evaluates all LLM-generated proposals against formal Invariant Contracts

before execution is permitted. This bifurcated design — probabilistic intelligence above, deterministic governance below — satisfies the reproducibility, auditability, and regulatory requirements of production fintech infrastructure. The system is patent-pending under U.S. Provisional 64/114,746 and U.S. Patent Application 19/693,343.

Five architectural claims define this system: entity-backed persistent neural topology (NeuralNode records that survive restarts and remain queryable); LLM-as-compute replacing mathematical propagation; deterministic governance (JasperOS) validating all LLM output before execution; self-healing intelligence across 150 or more fintech applications; and multi-LLM comparative confidence scoring under invariant-contract filtering.

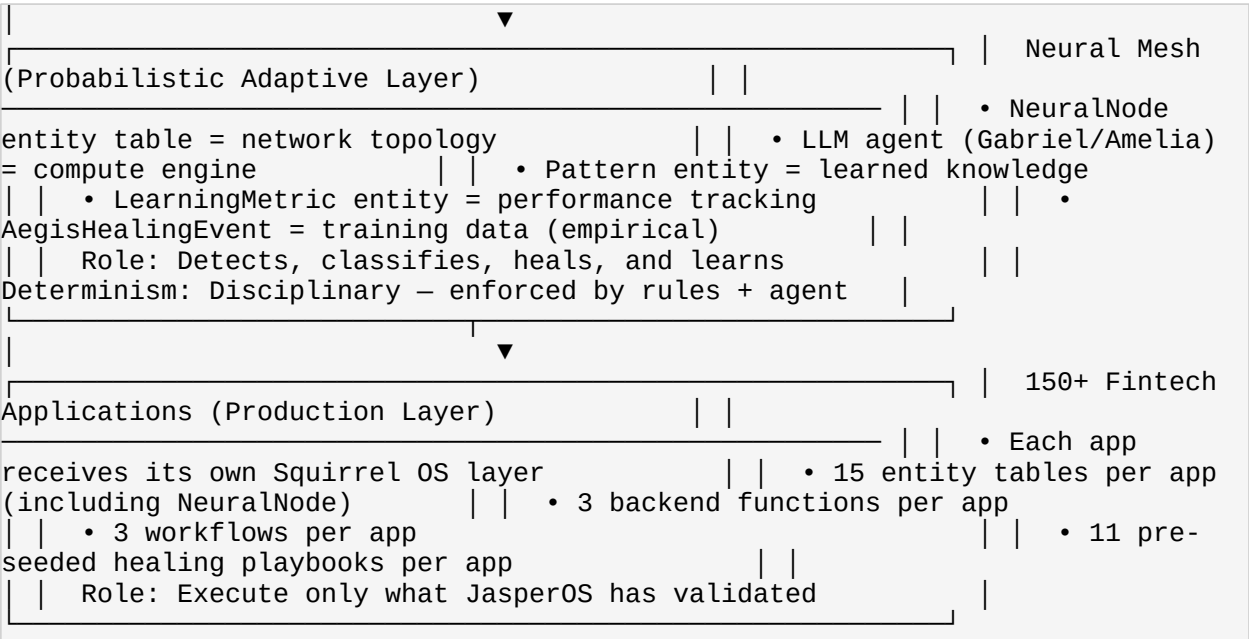
This document is the technical companion to the JasperOS HOWTO Manual. Together they describe the two-layer system: deterministic governance (JasperOS) and adaptive intelligence (Neural Mesh / Squirrel OS).

2. Architecture Overview

2.1 System Hierarchy

The Neural Mesh / JasperOS system is organized across three distinct tiers, each with a defined role, determinism model, and responsibility boundary. Tier 1 (JasperOS) governs all decisions deterministically through runtime-enforced contracts and an append-only state ledger. Tier 2 (Neural Mesh) operates probabilistically, using LLM agents to detect anomalies, classify patterns, select healing procedures, and adjust learned weights. Tier 3 comprises the 150 or more fintech applications, each provisioned with a full Squirrel OS layer and authorized to execute only actions that have been validated by Tier 1.

(Deterministic Governance Layer)		JasperOS	
Contracts evaluate at every boundary		• Invariant	
chained execution history		• State Ledger records hash-	
variance		• 12 concurrent LLM connections filtered for	
• Dispatch Graph enforces typed Signal routing			
• Epoch lifecycle: OPEN → ACTIVE → CLOSING → CLOSED			
• Role: Validates, audits, and governs all decisions			
Determinism: Structural – enforced by runtime			



2.2 Key Architectural Decisions

Decision	Rationale
LLM as compute engine	Enables adaptive reasoning over mathematical computation — the mesh handles novel anomaly types without retraining
Entity table as topology	Database-backed network structure allows persistence, querying, and cross-app replication
Deterministic governance	Financial infrastructure requires reproducibility — LLM variance is caught before execution
Bifurcated proposal/validation	LLMs propose actions; JasperOS validates against invariant contracts before execution
12 concurrent LLM connections	Comparative analysis across multiple probabilistic engines for confidence scoring

3. Data Model

3.1 NeuralNode Entity Schema

The NeuralNode entity is the fundamental unit of the neural mesh. Each record represents a single neuron in the network, stored persistently in the application database.

Field	Type	Description
layer	number	Network layer (1 = input, 2+ = hidden/deep, N = output/terminal)
weight	number	Connection strength (0.0-1.0), adjusted during learning cycles
connections	array	References to downstream nodes (e.g., ["L2-N1", "L2-N3"])
activation_count	number	Number of times this node has fired
pattern_type	string	Functional classification (e.g., input_heartbeat, hidden_pattern_match)
learning_rate	number	Rate of weight adjustment per learning cycle (0.01-0.08, increases with depth)
last_activated	datetime	Timestamp of most recent activation

3.2 Layer Structure: Starter Mesh (31 Nodes)

The initial mesh deployed on 25 July 2026 contains 31 nodes across 5 layers. This is a structural proof of concept — the full design targets 50,000 layers. Layer depth correlates with learning rate: deeper layers adjust faster, reflecting the greater uncertainty of output-layer decisions.

Layer 1 — Input Sensors (8 Nodes)

Node	Pattern Type	Weight	Connections	Learning Rate
L1-N1	input_heartbe	0.95	L2-N1, L2-N2,	0.01

Node	Pattern Type	Weight	Connections	Learning Rate
	at		L2-N3	
L1-N2	input_latency	0.92	L2-N2, L2-N4	0.01
L1-N3	input_error_rate	0.88	L2-N1, L2-N5	0.01
L1-N4	input_token_usage	0.94	L2-N3, L2-N6	0.01
L1-N5	input_memory	0.90	L2-N4, L2-N7	0.01
L1-N6	input_cpu	0.97	L2-N5, L2-N8	0.01
L1-N7	input_pqc_status	0.93	L2-N6, L2-N9	0.01
L1-N8	input_anomaly_count	0.91	L2-N7, L2-N10	0.01

Layer 2 — Hidden Processing (8 Nodes)

Node	Pattern Type	Weight	Connections	Learning Rate
L2-N1	hidden_pattern_match	0.78	L3-N1, L3-N2	0.02
L2-N2	hidden_trend_detect	0.82	L3-N1, L3-N3	0.02
L2-N3	hidden_anomaly_classify	0.75	L3-N2, L3-N4	0.02
L2-N4	hidden_severity_eval	0.80	L3-N2, L3-N5	0.02
L2-N5	hidden_root_cause	0.85	L3-N3, L3-N6	0.02
L2-N6	hidden_blast_radius	0.73	L3-N4, L3-N7	0.02
L2-N7	hidden_playbook_match	0.79	L3-N5, L3-N8	0.02
L2-N8	hidden_confidence_score	0.77	L3-N6, L3-N9	0.02

Layer 3 — Deep Analysis (7 Nodes)

Node	Pattern Type	Weight	Connections	Learning Rate
L3-N1	deep_isolation_strategy	0.65	L4-N1, L4-N2	0.03
L3-N2	deep_healing_selection	0.70	L4-N1, L4-N3	0.03
L3-N3	deep_verification_logic	0.68	L4-N2, L4-N4	0.03
L3-N4	deep_escalation_eval	0.72	L4-N3, L4-N5	0.03
L3-N5	deep_quantum_threat	0.66	L4-N4, L4-N6	0.03
L3-N6	deep_cross_app_cascade	0.64	L4-N5, L4-N7	0.03
L3-N7	deep_learning_route	0.69	L4-N6, L4-N8	0.03

Layer 4 — Output Decisions (5 Nodes)

Node	Pattern Type	Weight	Connections	Learning Rate
L4-N1	output_heal_action	0.55	L5-N1	0.05
L4-N2	output_escalate_action	0.58	L5-N1, L5-N2	0.05
L4-N3	output_log_action	0.60	L5-N2	0.05
L4-N4	output_alert_action	0.53	L5-N1, L5-N3	0.05
L4-N5	output_proposal_action	0.56	L5-N2, L5-N3	0.05

Layer 5 — Terminal Results (4 Nodes)

Node	Pattern Type	Weight	Connections	Learning Rate
L5-N1	terminal_healing_result	0.45	—	0.08
L5-N2	terminal_escalation_result	0.48	—	0.08
L5-N3	terminal_learning_extract	0.42	—	0.08
L5-N4	terminal_pattern_update	0.44	—	0.08

3.3 Supporting Entities

Entity	Role in Mesh	Relationship to NeuralNode
AegisHealingEvent	Training data — empirical outcomes from healing actions	Feeds into learning cycles; each event fires nodes and adjusts weights
Pattern	Learned knowledge — recognized anomaly signatures	Created/updated by the mesh during learning; confidence_score reflects mesh output
LearningMetric	Performance tracking — trend data over time	Records weight adjustments, success rates, resolution times
AegisPlaybook	Healing procedures — the "motor skills" the mesh selects	Selected by Layer 2 (playbook_match) and Layer 3 (healing_selection) nodes
AegisAnomaly	Input stimuli — detected anomalies that trigger the mesh	Feeds into Layer 1 input nodes based on anomaly_type
Insight	Derived intelligence — higher-order conclusions from patterns	Generated when occurrence_count exceeds threshold
SelfImprovementProposal	Evolution proposals — suggested changes to the system	Generated from insights; feeds back into playbook updates

4. Compute Model

4.1 LLM-as-Compute-Engine

In a conventional neural network, forward propagation is a sequence of matrix multiplications followed by non-linear activation functions — a purely mathematical process. In the Neural Mesh Architecture, this mathematical process is replaced by LLM reasoning. The LLM agent receives the current state of the network — activated nodes, their weights, and their connections — and determines which downstream nodes should fire and with what intensity. This substitution enables the mesh to handle novel, previously unseen anomaly types without retraining, because the LLM's language understanding generalizes across input types.

Forward Propagation (LLM-Driven)

1. Anomaly detected → AegisAnomaly record created
2. Layer 1 input nodes fire based on anomaly metrics (heartbeat, latency, CPU, etc.)
3. LLM agent reads activated input nodes and their current weights
4. LLM evaluates which Layer 2 nodes should fire (pattern matching, classification)
5. LLM continues propagation through Layers 3-5
6. Terminal node determines action: heal, escalate, log, alert, or propose

Weight Adjustment (LLM-Driven)

7. After healing completes, outcome is recorded as AegisHealingEvent
8. LLM agent runs the pattern-learning skill
9. For each fired node: weight increases by learning_rate on success; decreases on failure
10. activation_count increments
11. Pattern entity updated (occurrence_count, confidence_score, last_seen)
12. LearningMetric recorded (trend, comparison_to_previous)

4.2 Comparison: Traditional Neural Network vs. This Architecture

Property	Traditional Neural Network	This Architecture
Topology	In-memory tensor matrices	Database entity records (NeuralNode table)
Forward pass	Matrix multiplication + activation function	LLM reasoning over node states and weights
Backpropagation	Chain rule + gradient descent	LLM evaluates outcomes and adjusts weights empirically
Loss function	Mathematical (MSE, cross-entropy)	Empirical success rate (heals succeeded / total attempts)
Learning rate	Scalar multiplied by gradient	Scalar added/subtracted from weight (0.01–0.08)
Persistence	Ephemeral (lost on restart)	Permanent (database records survive restarts)
Reproducibility	Deterministic given same inputs	Non-deterministic (LLM variance) — governed by JasperOS
Novel input handling	Requires retraining	LLM reasons about novel anomaly types without retraining
Scale at 50K layers	Billions of parameters	50,000 database records (feasible on standard infrastructure)

4.3 Honest Classification

This architecture is most accurately described as an **adaptive pattern recognition mesh with LLM-driven compute**. It is a legitimate and novel class of machine intelligence — not a conventional mathematical neural network, and not a marketing rebranding of an existing construct. The distinction is precise and material: the compute substrate is linguistic reasoning, the topology is relational data, and the learning signal is empirical outcome, not calculus. Technical readers should understand the following classification:

- ✓ A legitimate adaptive learning system (weights adjust based on empirical outcomes)

- ✓ A persistent neural topology (survives restarts, queryable, and replicable across apps)
- ✓ A self-improving system (gets smarter with each healing event)
- ✓ Patent-pending as a novel architectural class (see Section 9)
- ✗ Not a mathematical neural network (no matrix math, no gradient descent)
- ✗ Not deep learning (no backpropagation through calculus)
- ✗ Not guaranteed reproducible (LLM variance — mitigated by JasperOS deterministic governance)

5. Learning Protocol

5.1 Learning Cycle (Pattern-Learning Skill)

The learning cycle is triggered after every successful AegisHealingEvent or manually by the operator.

```

LEARNING CYCLE 1. READ healing events      Query AegisHealingEvent records
(result = "success")      Extract: anomaly_type, playbook_id, steps_executed,
outcome 2. EXTRACT anomaly signature      anomaly_type → determines which
Layer 1 input nodes fire      root_cause → determines which Layer 2
root_cause nodes activate      playbook_id → determines which Layer 2
playbook_match nodes activate      outcome → determines weight adjustment
direction 3. SEARCH existing patterns      Query Pattern records for matching
anomaly_types      If found: increment occurrence_count, update last_seen,
adjust confidence_score      If not found: create new Pattern
(occurrence_count=1, confidence_score=0.5) 4. FIRE neural nodes      Layer 1:
input nodes fire based on anomaly metrics      Layer 2: hidden nodes fire based
on pattern match + classification      Layer 3: deep nodes fire based on
isolation/healing strategy      Layer 4: output nodes fire based on action
decision      Layer 5: terminal nodes fire based on result 5. ADJUST weights
For each fired node:      weight = weight ± (learning_rate × outcome_signal)
outcome_signal = +1 (success) | -1 (failure)      Clamp weight to [0.0, 1.0]
6. RECORD learning metrics      Create LearningMetric: resolution_time,
success_rate, auto_heal_rate      Update Pattern: confidence_score based on
historical success rate 7. GENERATE insights (if occurrence_count > 5)
Create Insight record from pattern analysis      If recurrence is high →
generate SelfImprovementProposal 8. MARK events as learned      Update
AegisHealingEvent: learning_extracted = true

```

5.2 First Learning Cycle Results (25 July 2026)

Metric	Value
Healing events analyzed	50
Patterns extracted	3
Neural nodes fired	14 (across all 5 layers)
Weight adjustments	14 (all positive — 100% success rate)
Dominant anomaly type	heartbeat_miss (50 occurrences)
Secondary anomaly type	integration_degraded (297 occurrences in playbook data)
Tertiary anomaly type	cpu_spike (3 occurrences)

5.3 Extracted Patterns

Pattern ID	Name	Occurrences	Confidence	Recommended Playbook
PAT-001	Heartbeat Miss — Monitoring Loop Stall	50	0.95	PB-008
PAT-002	Integration Failover — Upstream Provider Degradation	297	0.98	PB-011
PAT-003	CPU Spike — Neural Mesh Training Load	3	0.85	PB-002

6. Governance Integration

6.1 The Proposal-Validation Model

The system operates on a strict bifurcation between proposal (probabilistic) and validation (deterministic). The Neural Mesh layer proposes healing actions based on learned patterns and LLM reasoning; JasperOS independently evaluates every proposal against formal Invariant Contracts before any action is committed to execution. No healing action reaches the production application layer without passing

JasperOS validation. This design ensures that LLM variance — an inherent property of probabilistic language models — cannot produce unsafe, non-compliant, or out-of-contract behavior in a live fintech environment.

PROPOSE (Probabilistic Layer)

Neural Mesh detects anomaly → LLM evaluates pattern → LLM selects playbook → LLM executes healing steps → AegisHealingEvent recorded → Weights adjusted → Pattern updated → Learning metric recorded

VALIDATE (Deterministic Layer)

JasperOS Invariant Contract evaluates: → Was the anomaly_type exactly matched? → Was the confidence_score above threshold? → Was the playbook execution within parameters? → Were any invariant boundaries violated? → Was PQC validation performed (if crypto operation)? → Was sensitive data redacted in logs? If all checks pass → action committed to State Ledger
If any check fails → action rejected, escalated to human

6.2 Escalation Rules (Human-in-the-Loop)

The mesh escalates to a human operator when any of the following conditions are met:

Condition	Action
Healing attempt fails twice on same anomaly	Create PredictiveAlert (critical), escalate to operator
Critical-severity anomaly with no matching playbook	Create PredictiveAlert, log, escalate
PQC validation failure detected	Immediate escalation, halt crypto operations
SystemHealth critical for 3+ consecutive heartbeats	Escalate, trigger full system sweep
Confidence score below playbook threshold	Log as "detected", create PredictiveAlert, do NOT auto-heal

6.3 Multi-LLM Comparative Filtering

13. All 12 LLMs receive the same anomaly input simultaneously
14. Each produces an independent healing proposal
15. JasperOS Invariant Contracts evaluate all 12 proposals
16. Proposals that violate contracts are rejected

- 17. Remaining proposals are ranked by confidence score
- 18. Highest-confidence proposal that passes all contracts is executed
- 19. All 12 proposals and the selection rationale are recorded in the State Ledger

Over time, this creates a comparative confidence scoring system — the mesh learns which LLMs are most reliable for which anomaly types.

7. Deployment Model

7.1 Per-App Squirrel OS Template

Each app in the 150+ fintech ecosystem receives the full Squirrel OS layer — a standardized, fully provisioned operating environment.

Component	Count	Purpose
Entity tables	15	Full data model including NeuralNode
Backend functions	3	healthCheck, systemMetrics, ameliaHeartbeat
Workflows	3	Heartbeat Monitor, Daily Sweep, Anomaly Auto-Response
Playbooks	11	PB-001 through PB-011 (pre-seeded healing procedures)
Skills	4	heartbeat-check, full-system-sweep, anomaly-response, pattern-learning
Seed agents	4	Initial OrchestratorAgent records
Seed nodes	4	Initial OrchestratorNode records
Policy rules	11	Core operating directives

7.2 Current Deployment Status (as of 25 July 2026)

Metric	Value
Total apps identified	100+
Apps with Squirrel OS deployed	37
Apps in standby mode	36 (workflows paused)
Active hypervisor instance	1 — Jasper / Squirrel OS
Neural mesh populated	1 (Gabriel)
NeuralNode schema available	37 (all deployed apps)
Healing events (Jasper instance)	297
Healing success rate	100%

7.3 Scaling the Neural Mesh

- 20. **Seed phase** — Deploy the 31-node starter mesh to each app via backend function
- 21. **Activation phase** — Enable pattern-learning skill on each app
- 22. **Specialization phase** — Each app's mesh learns domain-specific patterns (tokenization, settlement, trading, etc.)
- 23. **Federation phase** — All meshes report patterns up to Jasper for cross-app learning
- 24. **Optimization phase** — ARETE drives recursive optimization across all meshes

8. Security and Compliance

8.1 Post-Quantum Cryptography (PQC) Integration

Every app's Squirrel OS layer includes domain-specific PQC algorithms:

- **CRYSTALS-Dilithium** — digital signature
- **Kyber-1024** — key encapsulation

- **SPHINCS+-256f** — hash-based signature

The neural mesh's Layer 3 includes the `deep_quantum_threat` node (L3-N5, weight 0.66), which evaluates quantum vulnerability in every healing decision path.

8.2 Data Redaction Policy

- No PII, wallet addresses, transaction amounts, or private keys are logged
- AegisHealingEvent records contain only structural and operational metadata
- Alerts and logs use redacted summaries
- Compliant with Squirrel OS Policy Rule #8

8.3 Audit Trail

Every healing action creates a permanent AegisHealingEvent record containing:

- Anomaly reference
- Playbook reference
- Steps executed
- Outcome (success / failure)
- Duration
- Learning extracted flag
- Agent and node references

These records serve dual purpose: empirical training data for the neural mesh, and the regulatory audit trail for fintech operations.

9. Patent Alignment

Patent	Coverage	Neural Mesh Role
U.S. Provisional 64/114,746	Deterministic method for distributed execution	JasperOS governance layer validates all mesh output

Patent	Coverage	Neural Mesh Role
U.S. Patent Application 19/693,343	Multi-rail settlement with deterministic oversight	Neural mesh provides self-healing intelligence for settlement infrastructure

The Neural Mesh Architecture extends these patent claims through four novel contributions:

- 25. **Adaptive learning from empirical outcomes** — weights adjust from real healing results, not mathematical loss functions
- 26. **Entity-backed persistent network topology** — the neural network is a queryable, replicable database, not an ephemeral tensor
- 27. **LLM-driven compute replacing mathematical propagation** — reasoning replaces matrix math
- 28. **Multi-LLM comparative confidence scoring under deterministic governance** — 12-engine consensus filtered by invariant contracts

10. Roadmap

Phase	Description	Status
Starter mesh	31 nodes across 5 layers deployed on Gabriel	✓ Complete — 25 July 2026
First learning cycle	Pattern extraction and neural node firing	✓ Complete — 25 July 2026
Mesh propagation	Deploy NeuralNodes to 36 additional apps	⌚ Pending
Workflow reactivation	Resume heartbeat, sweep, and anomaly triggers	⌚ Pending (standby mode)
ARETE integration	Recursive mesh optimization engine	⌚ Pending
50K-layer expansion	Scale to full neural mesh design	🕒 Future

Phase	Description	Status
Cross-app federation	Meshes share patterns via Jasper hypervisor	 Future
Self-improvement loop	Automated proposal → approval → deployment cycle	 Future

Appendix A — Entity Schemas

NeuralNode

```
{  "type": "object",  "properties": {    "layer": { "type": "number" },    "weight": { "type": "number" },    "connections": { "type": "array", "items": { "type": "string" } },    "activation_count": { "type": "number" },    "pattern_type": { "type": "string" },    "learning_rate": { "type": "number" },    "last_activated": { "type": "string", "format": "date-time" }  } }
```

Pattern

```
{  "type": "object",  "properties": {    "pattern_id": { "type": "string" },    "name": { "type": "string" },    "anomaly_types": { "type": "array", "items": { "type": "string" } },    "common_root_causes": { "type": "object" },    "confidence_score": { "type": "number" },    "occurrence_count": { "type": "number" },    "first_seen": { "type": "string" },    "last_seen": { "type": "string" },    "recommended_playbook_id": { "type": "string" },    "auto_heal_enabled": { "type": "boolean" },    "source_domain": { "type": "string" },    "status": { "type": "string" }  } }
```

LearningMetric

```
{  "type": "object",  "properties": {    "metric_name": { "type": "string" },    "value": { "type": "number" },    "trend": { "type": "string" },    "comparison_to_previous": { "type": "string" },    "period": { "type": "string" },    "recorded_at": { "type": "string", "format": "date-time" }  } }
```

Appendix B — Glossary

Term	Definition
Neural Mesh	The entity-backed adaptive learning network described in this document
NeuralNode	A single neuron record in the mesh, stored as a database entity
LLM-as-Compute	Using a Large Language Model as the forward propagation and weight adjustment engine
Deterministic Governance	JasperOS layer that validates all probabilistic decisions before execution
Invariant Contract	Formal predicate evaluated at every system boundary by JasperOS
State Ledger	Append-only, hash-chained record of all execution decisions
AegisPlaybook	Immutable healing procedure matched to anomaly types
AegisHealingEvent	Audit trail record of every healing action; primary training data for the mesh
Pattern	Recognized anomaly signature extracted from healing events
Squirrel OS	The operating system layer deployed across the fintech app ecosystem
Jasper	The top-level AI Orchestration Supervisor (hypervisor)
Amelia	The self-healing engine with 50K neural mesh design
Gabriel	The first deployed neural mesh instance (activated 25 July 2026)
ARETE	AI orchestrator identified as the recursive mesh optimization engine
PQC	Post-Quantum Cryptography
Base44	The deployment platform hosting the Squirrel OS / JasperOS ecosystem

© 2026 Leon Calvin Long II. Patent Pending.

U.S. Provisional 64/114,746 · U.S. Patent Application 19/693,343

This document is the technical companion to the JasperOS HOWTO Manual.
Together they describe the two-layer system: deterministic governance (JasperOS)
and adaptive intelligence (Neural Mesh / Squirrel OS).
Suitable for: [GitHub repository](#) · [Zenodo deposit](#) · [Ecosystem partner onboarding](#).